Electricity Production and Supply Prediction

Bolem Siva Rama Lalitha Lakshmi

Registration No. 2023001420

Kukutam Bharath Reddy

Registration No. 2023006001

School of Science,

GITAM Deemed to University,

Hyderabad Campus

M. Sc. Data Science

2025

Electricity Production and Supply Prediction

By

Bolem Siva Rama Lalitha Lakshmi
Reg No: 2023001420

Kukutam Bharath Reddy
Reg No: 2023006001

Submitted in Partial fulfillment of the Requirements for the Degree of Master of Science in Data Science at GITAM Deemed to be University

Under Supervisor

Dr. BM Naidu

School of Sciences

GITAM Deemed To Be University

Rudraram, Hyderabad Campus

April 2025

Declaration and Approval

Declaration

I certify that this dissertation/Major Project is my own original work and has not been previously submitted or approved for the award of a degree at this or any other university. To the best of my knowledge, it does not include any material authored or published by others, except where appropriate citations and references are given.

©This dissertation may not be reproduced in any form without prior authorization from the author and GITAM University.

Student's Name: Bolem Siva Rama Lalitha Lakshmi
Student's Name: Kukutum Bharath Reddy

Approval

The dissertation of Your Name was reviewed and approved for examination by the following:
Dr. BM Naidu,
Department of Mathematics,
School of Sciences,
GITAM University,
Hyderabad Campus
External Expert,

Principal,
School of Science,
GITAM University

Acknowledgements

I would like to express my sincere gratitude to my project supervisor, Dr.BM Naidu sir, for his invaluable guidance, constant support, and encouragement throughout the course of my major project. His expertise and insights have played a crucial role in shaping this work, and I am truly grateful for his mentorship.

I extend my heartfelt thanks to Dr. Motahar Reza, Principal, School of Science, GITAM University, Hyderabad, for providing a stimulating academic environment and unwavering support. I am also deeply grateful to Dr. Abhijit Patil, Project Coordinator, for his assistance, constructive feedback, and motivation, which have been instrumental in the successful completion of this project.

I would also like to acknowledge Prof. D. S. Rao, Pro Vice-Chancellor, GITAM Hyderabad Campus, for his continuous support and for fostering an environment that encourages academic excellence and research.

I am grateful to my professors and mentors, whose teachings and guidance have been fundamental in shaping my knowledge and skills in Data Science. Their support has been invaluable in my academic journey.

Furthermore, I am profoundly thankful to my parents and family for their unconditional love, encouragement, and sacrifices, which have been my greatest source of strength. A special note of appreciation goes to my friends for their continuous support, insightful discussions, and encouragement, making this journey more enriching and memorable.

Finally, I express my gratitude to everyone who has contributed to my learning experience at the School of Science, GITAM University, Hyderabad. Their support has been instrumental in the successful completion of this project.

BSR Lalitha Lakshmi and K Bharath Reddy

M.Sc. Data Science

School of Science, GITAM University, Hyderabad

Abstract

Forecasting electricity production and supply is fundamental for power grid supervision, optimizing resources, and energy sustainability. Times series forecasting models are advantageous for this kind of time-dependent forecasting. This project explores forecasting models such as ARIMA, SARIMA, SARIMAX, and a few neural network models such as LSTM and Deep Neural Networks.

By allowing for the data on energy production and other influential factors on energy like weather patterns, the project involves developing modified data by concatenating the existing datasets and performing Principal Component Analysis to progress the model's accuracy and adaptability. The project also aims to learn the trends, and seasonality captured within the data. Moreover, pre-processing the collected data before implementing any model helps in better model building.

Traditional forecasting models consist of steps like checking for stationarity, re-sampling the data, model building, and fitting the model, whereas in neural network models the data will be passed through different layers of the network following its architecture. On a comparative study of the results from both forecasting and neural network models, this project provides expensive insights that can be used for the decision-making of demand prediction, load balancing, etc. This work contributes to the development of smart and sustainable energy systems which also help the environment.

Table of Contents

List of Tables

List of Figures

1 Introduction

1.1 Background

Forecasting electricity production and supply has become a crucial challenge for energy providers and grid operators in recent years due to factors like climate unpredictability, the integration of renewable energy sources, and rising electricity demand. Better load balancing, resource allocation, energy pricing, and environmental sustainability are all provided by effective forecasting.

Because time series models like ARIMA, SARIMA, and SARIMAX can capture temporal patterns and identify trends and seasonality. When dealing with non-linear and multi-factor influenced data like temperature and humidity. Advanced forecasting methods, especially those based on machine learning and deep learning, have been developed to address issues like non-linearity.

Dynamic energy systems can benefit from models like Deep Neural Networks (DNNs) and Long Short-Term Memory (LSTM) networks, which can learn non-linear correlations and long-term dependencies in data. Even with the advancement of these sophisticated techniques, a comparison of their performance to conventional models in the context of electricity forecasting is still required.

By merging several datasets, using dimensionality reduction methods like Principal Component Analysis (PCA), and assessing the forecasting precision of both statistical and deep learning models.

The project's objectives are to influence future forecasting techniques, offer insightful information on model performance, and aid in the development of more sustainable energy systems.

1 Introduction

1.2 Problem Statement

Reducing expenses, fulfilling energy demand, and improving grid management all depend on accurate forecasting of electricity production and supply. The objective of this research is to construct a forecasting model in a comparison study employing time series approaches like ARIMA, SARIMA, SARIMAX, and techniques like neural networks such as LSTM, DNN. Prior research on electricity forecasting has mostly concentrated on conventional time series techniques like ARIMA and SARIMA, which work well for stationary and linear data but poorly for non-linear and dynamic patterns. Power producing firms will be able to optimize planning, pricing, and renewable energy initiatives with the help of improved forecasting accuracy, improved grid management, and support for sustainable energy systems.

1.3 Research Objectives

The primary goal of the project is to build and evaluate the forecasting models for electricity production and supply by utilizing both traditional time series techniques and advanced neural network architectures. The performance of these models will be compared based on the prediction accuracy.

- ✓ To gather and interpret energy related data along with influential variables such as weather conditions.
- ✓ To clean, preprocess, and merge various datasets to create a comprehensive and usable dataset.
- ✓ To use dimensionality reduction methods like Principal Component Analysis (PCA) to identify the most impactful features for accurate forecasting.

- ✓ To design and implement a range of models including ARIMA, SARIMA, SARIMAX, Long Short-Term Memory (LSTM), and Deep Neural Networks (DNN).
- ✓ To assess model performance using evaluation metrics such as Root Mean Square Error (RMSE), Mean Squared Error (MSE), and Mean Absolute Error (MAE) and to perform comparative analysis.
- ✓ To generate future prediction visualizations that support effective energy management, intelligent and eco-friendly energy infrastructures.

1.4 Research Questions

i. How efficient are time series models in forecasting electricity production and supply based the data considered?
ii. Can deep learning models perform well than forecasting models?
iii. What is the use of including /considering external factors like weather and how impactful it is, on energy production?
iv. How dimensionality reduction helps in improving the performance of the model.
v. Which technique performed well among all according the comparative study?
vi. Why to check the stationarity when working with time-series techniques?

1 Introduction

1.5 Justification

With the increasing electricity demand, unevenness of energy consumption patterns due to the external factors like weather and humidity, accurate forecasting of electricity production and supply has turn out to be significant. Energy generating companies need consistent forecasting tools to make sure the efficient energy distribution, reduce losses, plan resource provision, and maintain grid constancy.

Long-established forecasting methods like ARIMA, SARIMA, and SARIMAX are extensively used due to their cleanness and interpretability. On the other hand, these models regularly fail when dealing with non-stationary, non-linear and multi-factor dependent data, which is universal in real-world energy systems. This initiated the need to consider and explore more advanced methods to overcome the drawbacks.

So, we considered Deep leaning architectures such as Long Short-Term Memory (LSTM) and Deep Neural Networks (DNN), to check whether they can handle the complicated relationships in time-dependent datasets. Combining these models with the dimensionality reduction methods like Principal Component Analysis (PCA) can help to improve the accuracy of the neural network model.

By performing a comparative analysis of these models on real-world energy data, we can understand the best model to work with when the data is time-dependent.

The insights obtained can help the power generating companies to make knowledgeable decisions about prediction of energy demand, storage, and load balancing. This also helps in contributing to the improvement of smart, efficient, sustainable energy systems.

1 Introduction

1.6 Scope

The project focuses on forecasting energy production and supply using both traditional time series techniques and advanced deep learning models.

In-scope:

- Data collection and Preprocessing:
  Gathering historical data associated to energy production along with the influencing factors like weather conditions. Then, prepare the data for the further model building through cleaning, analyzing, merging etc.
- Dimensionality Reduction:
  Implementing techniques like Principal Component Analysis (PCA) helps in extracting relevant features from the original dataset.
- Model Building:
  Developing forecasting models such as ARIMA, SARIMA, and SARIMAX, and Deep Learning models such as Long Short-Term Memory (LSTM), and Deep Neural Networks (DNN).
- Comparative Analysis:
  Evaluating the performance of all implemented models and compare them.
- Insights:
  Derive insights from model outcomes to help companies in taking informed-decisions, prediction of demand, load balancing, etc.

Out-scope:

- Deploying of forecasting systems in a live grid systems.
- Integration of IoT or smart meter technologies in forecasting.
- Working on other country electricity production and supply data

1 Introduction

1.7 Limitations

- Data Quality: Missing or inconsistent data may reduce model accuracy.
- Data Complexity: Quantifying the impact of factors like weather and seasonal trends could be difficult.
- Overfitting: Risk of overfitting models to historical data, leading to poor future predictions.
- Computational Demands: Advanced models like Neural Networks and LSTM require significant computational power.
- Real-time deployments are beyond the scope of this project.
- Models are evaluated on a specific dataset, so the findings may not generalize across all regions or energy systems.

2 Literature Review

2.1 Study 1

Title: "Time-series forecasting with deep learning: a survey" Publication Year: 2021 Authors: Bryan Lim and Stefan Zohren The article "Time-series forecasting with deep learning: a survey" by Bryan Lim and Stefan Zohren gave a clean overview of how deep learning techniques are revolutionizing the practice of time-series forecasting. It starts by describing the shortage of standard statistical models such as ARIMA and in capturing complicated temporal dependencies. They discussed the application of these models in diverse domains like finance, energy, transportation, and healthcare. Main challenges covered include preprocessing data, scalability, interpretability, and the capacity to deal with uncertainty. The survey also highlights increasing popularity for hybrid architectures that blend deep learning with probabilistic frameworks.

2.2 Study 2

Title: "Electricity demand estimation using ARIMA forecasting model" Publication Year: 2023 Authors: S. Sharma and S. K. Mishra The article "Electricity demand estimation using ARIMA forecasting model" by S. Sharma and S. K. Mishra discussed the application of the ARIMA (AutoRegressive Integrated Moving Average) model in forecasting electricity demand. The authors elaborated the need for precise electricity demand forecasting to manage energy and plan policies effectively. The ARIMA model is selected for its capability to model linear and stationary time-series data. The article talks about the methodology of model selection, parameter tuning, and performance assessment using historical electricity consumption data. The research supports the worth of statistical models in energy forecasting while proposing potential for further enhancement xi ii through hybrid or machine learning-based techniques. This article is a point of reference for practitioners operating in the energy industry.

2 Literature Review

2.3 Study 3

Title: "Applying Probabilistic Forecasting of Multidimensional Time Series to Electricity Load Analysis" Publication Year: 2023 Authors: Wenxiang Yin; Wen Ye; Guang Liu. The article explores the ability of probabilistic forecasting methods to improve electricity load forecasting, when it involved multidimensional time series data. The research points enhance the accuracy of forecasting. The experimental outcomes demonstrate that the probabilistic models perform better than the conventional methods with respect to both accuracy and dependability.

2.4 Study 4

Title: “ARIMA Time Series Modeling and Forecasting of Enterprise Electric Energy Consumption” Authors: Yizhuo Liu; Weijia Sun Publication Year: 2023 The discussed the efficiency of the ARIMA model in forecasting electric energy consumption in enterprises. The authors emphasize the increasing demand for precise energy forecasting to facilitate smart grid development and effective energy management. They outline a step-by-step process including data preprocessing, stationarity testing, and parameter adjustment of the model. The paper concludes by proposing that hybrid models or deep learning methods might supplement ARIMA in subsequent applications. Generally, the research highlights the pragmatic utility of traditional statistical models in industrial energy planning.

2 Literature Review

2.5 Study 5

Title: "Charging load forecasting of electric vehicles based on VMD-SSA-SVR" Publication Year: 2023 Author(s): Y. Wu, P. Cong and Y. Wang, The paper "Charging Load Forecasting of Electric Vehicles Based on VMD-SSA-SVR" presents a hybrid approach involving Variational Mode Decomposition (VMD), Sparrow Search Algorithm (SSA), and Support Vector Regression (SVR) to enhance the accuracy of EV charging load predictions. VMD separates complicated time-series information, SSA searches for the best SVR parameters, and the two combine to handle volatility and nonlinearity in EV charging patterns.

3 Methodology

3.1 Development Methodology

Our project follows a comparative study for forecasting electricity production and supply using both statistical and deep learning models. The main aim of the project is to review the performance of the models like ARIMA, SARIMA, SARIMAX, LSTM, DNN by working on a same dataset.

3.1.1 Phase 1

Dataset collection & Pre-processing :

For this project we collected two datasets. One is related to energy production and supply of London City. The other is the weather dataset of the same city, which acts as the dependent factors for the energy production.

Energy Dataset:

- This is a daily basis dataset. It consists of columns: day, energy_median, energy_mean, energy_max, energy_count, energy_std, energy_sum, energy_min.
- The data is stored in about 112 csv files. We performed concatenation over all the 112 csv files to form a single dataframe.
- As we observed null values, dropped the rows with less number of null values and imputed with median with high count of null vales.
- After plotting the box plots for all the columns of the datasets, we found outliers and one unnecessary column, energy_count.

3 Methodology

- Dropped energy_count and handled outliers in IQR method.
- Chose day which is in datetime format as the index for the further steps.
- Saved the final energy dataframe into csv file in the local drive.

Weather Dataset:

- It contains the variables: 'temperatureMax', 'temperatureMaxTime', 'windBearing', 'icon', 'dewPoint', 'temperatureMinTime', 'cloudCover', 'windSpeed', 'pressure', 'apparentTemperatureMinTime', 'apparentTemperatureHigh', 'precipType', 'visibility', 'humidity', 'apparentTemperatureHighTime', 'apparentTemperatureLow', 'apparentTemperatureMax', 'uvIndex', 'time', 'sunsetTime', 'temperatureLow', 'temperatureMin', 'temperatureHigh', 'sunriseTime', 'temperatureHighTime', 'uvIndexTime', 'summary', 'temperatureLowTime', 'apparentTemperatureMin', 'apparentTemperatureMaxTime', 'apparentTemperatureLowTime', 'moonPhase'
- The dataset size is 882 row x 35 columns
- Loaded and preprocessed the data similarly.
- Handled null values by imputing with median as there are very less.
- Extracted day, time columns from the existed column.
- Performed encoding by creating dummies, then created new columns and concatenated to the original dataset.
- Dropped few unnecessary columns from the original dataset.

3 Methodology

- Chose day which is in datetime format as the index for the further steps
- Saved the final weather dataframe into csv file in the local drive.

Final Dataset:

- Loaded the previous prepared two datasets.
- Merged the 2 datasets, energy_data and weather_data into one dataset considering the day as the common column.
- Set index as the day column.
- As we already preprocessed the energy and weather datasets earlier before merging, no specific data preprocessing steps required at this stage.
- Saved the final dataframe into csv file in the local drive.
- Shape of the final dataset is 2948624 rows x 37 columns

Implementing PCA:

- We performed dimensionality reduction technique Principal Component Analysis (PCA).
- The total number of principal components, which are highly contributing are 17.
- Saved the final dataframe into csv file in the local drive.

3 Methodology

3.1.2 Phase 2

Model Building:

As part of model building, we developed and evaluated multiple forecasting models to predict the electricity production and supply.
We grouped the models into two categories:

1. Statistical models
2. Deep Learning models

We did comparative study which helps us to analyze the performance and sustainability of each model type in handling temporal data with external influencing variables like weather.

1. ARIMA (AutoRegressive Integrated Moving Average):
   - We imported all the required libraries and loaded the final dataset.
   - As the dataset is too large in size, as it is daily data, we considered re-sampling the dataset to Weekly mean.
   - Plotted the required time-series plots.
   - Performed ADF test to check the stationarity of the series.
   - Plotted Autocorrelation and Partial Autocorrelation (acf and pacf) graphs to find the p, d, q values.
   - Plotted different plots for better understanding.
   - Tried auto_arima to find the optimal p,d,q values by stepwise search for minimal AIC value.
   - Performed model fit. Tried different (p,d,q) values and considered one.
   - Performed future forecast based on the past data for next 12 weeks.
   - Plotted residuals to find the normality.
   - Our ARIMA model meets most conditions for a reliable time series model: - Residuals are mostly normal, independent, and

homoscedastic. Low errors (MAE, MSE, RMSE) indicate good fit. Stationarity is ensured, and parameters were chosen correctly.

2. SARIMA (Seasonal AutoRegressive Integrated Moving Average):
   - Imported all the required libraries and loaded the dataset.
   - As the dataset is too large in size as daily data, we considered re-sampling the dataset to Weekly mean.
   - Plotted the required time-series plots.
   - Performed ADF test to check the stationarity of the series.
   - Plotted Autocorrelation and Partial Autocorrelation (acf and pacf) graphs to find the p, d, q values.
   - Plotted different plots for better understanding.
   - Tried auto_arima to find the optimal p,d,q values by stepwise search for minimal AIC value. Added seasonality to check the trends with sarima
   - Performed model fit. Tried different (p,d,q, P, D, Q, s) values and considered one.
   - Performed future forecast based on the past data for next 12 weeks.
   - Plotted residuals to find the normality.
   - Seasonal decomposition.

3. SARIMAX:
   - Imported all the required libraries and loaded the dataset.
   - As the dataset is too large in size as daily data, we considered re-sampling the dataset to Weekly mean.
   - Plotted the required time-series plots.

3 Methodology

- Performed ADF test to check the stationarity of the series.
- Plotted Autocorrelation and Partial Autocorrelation (acf and pacf) graphs to find the p, d, q values.
- Plotted different plots for better understanding.
- Tried auto_arima to find the optimal p,d,q values by stepwise search for minimal AIC value.
- Built SARIMAX model with SARIMA + Exogenous variables.
- Selected the first 4 variables from PCA step as the exogenous variables.
- Performed model fit. Performed future forecast based on the past data for next 12 weeks.

4. LSTM (Long Short-Term Memory):
    - Imported all the required libraries and loaded the dataset.
    - Performed MinMaxScaling over the data range
    - Took the sequence length as 30.
    - Split the dataset into 2 sets training and testing as 80:20
    - Built LSTM model
    - Compiled the Model using adam optimizer and suitable metrics
    - Trained the model.

5. Deep Neural Network (DNN):
    - Imported all the required libraries and loaded the dataset.
    - Performed MinMaxScaling over the datarange
    - Took the sequence length as 30.
    - Split the dataset into 2 sets training and testing as 80:20

3 Methodology

- Built DNN model
- Compiled the Model using adam optimizer and suitable metrics
- Trained the model.
- Performed Predictions with the test dataset.

3.1.3 Phase 3

Model Evaluation:

- We performed all the models with same metrics and dataset to ensure effective comparison among the model performance.
- The performance metrics we used are:
    - Mean Absolute Error (MAE)
    - Mean Squared Error (MSE)
    - Root Mean Squared Error (RMSE)
- The comparative analysis on the performance of the models was conducted.

4 System Design and Architecture

The system we developed for the electricity production and supply prediction follows a structure, starts with data collection and ends with model deployment.

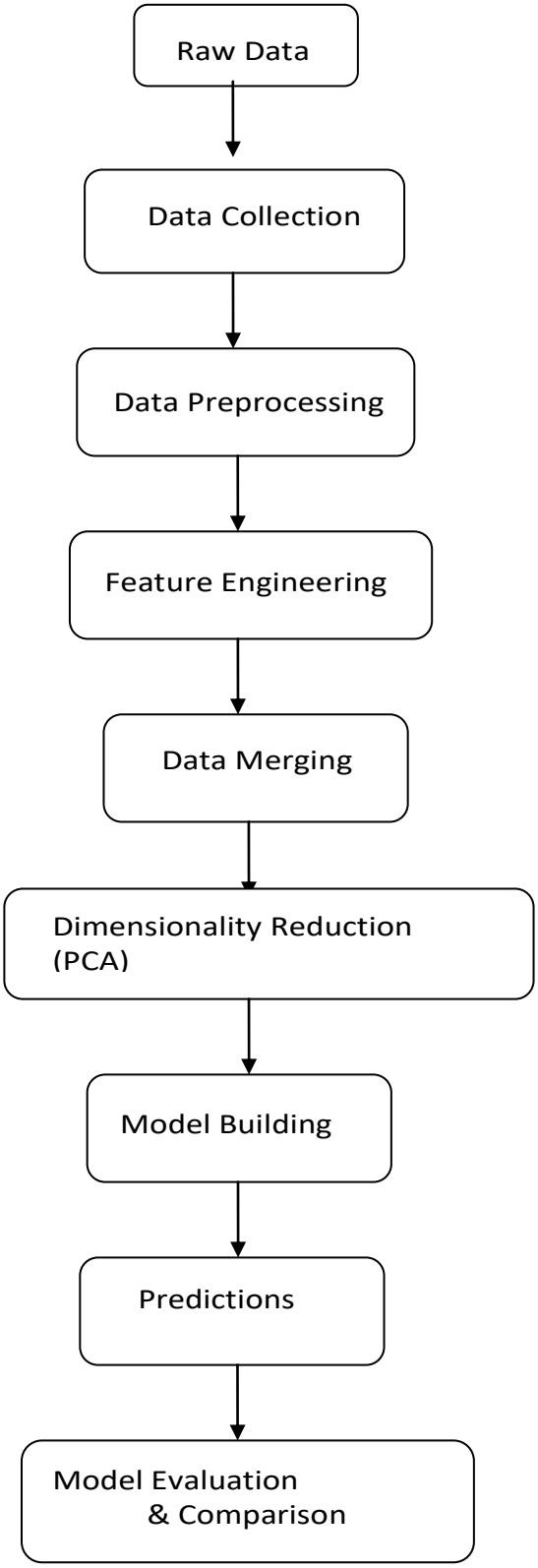
System Overview

We divided the entire system architecture into five different modules.

1. Data Collection
2. Data Preprocessing and Feature Engineering
3. Dimensionality reduction
4. Model Building and Training
5. Model Evaluation and Comparison

4 System Design and Architecture

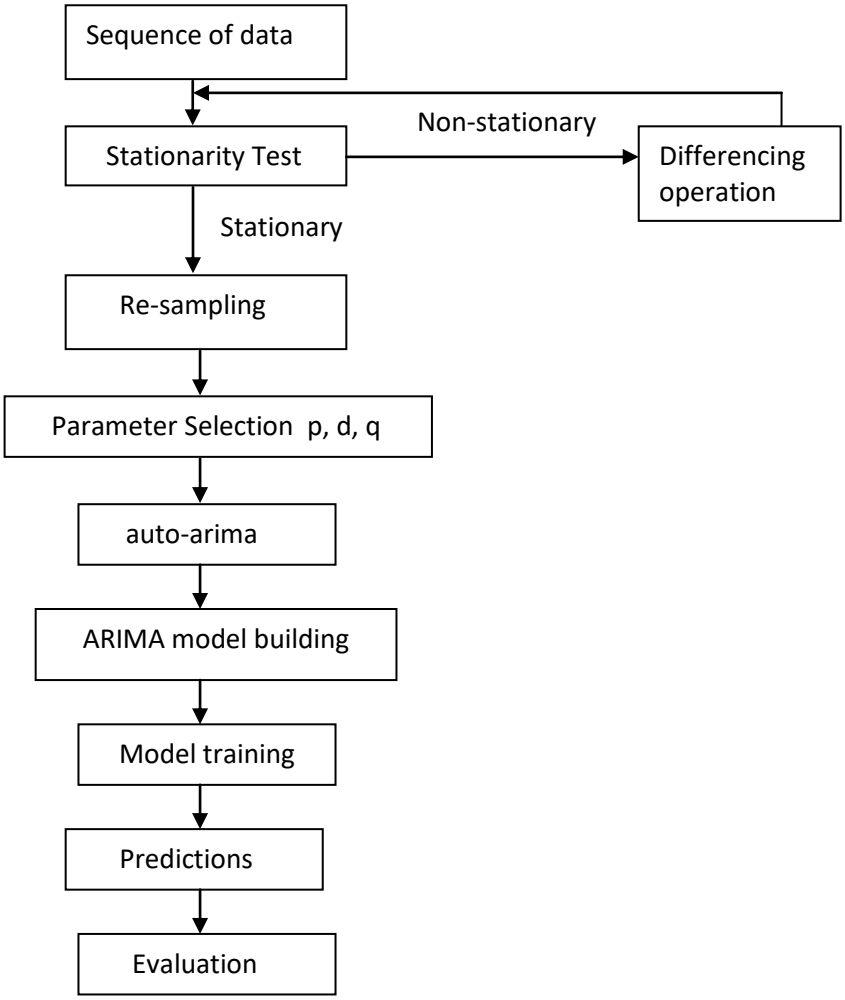
System Design and Architecture



4 System Design and Architecture

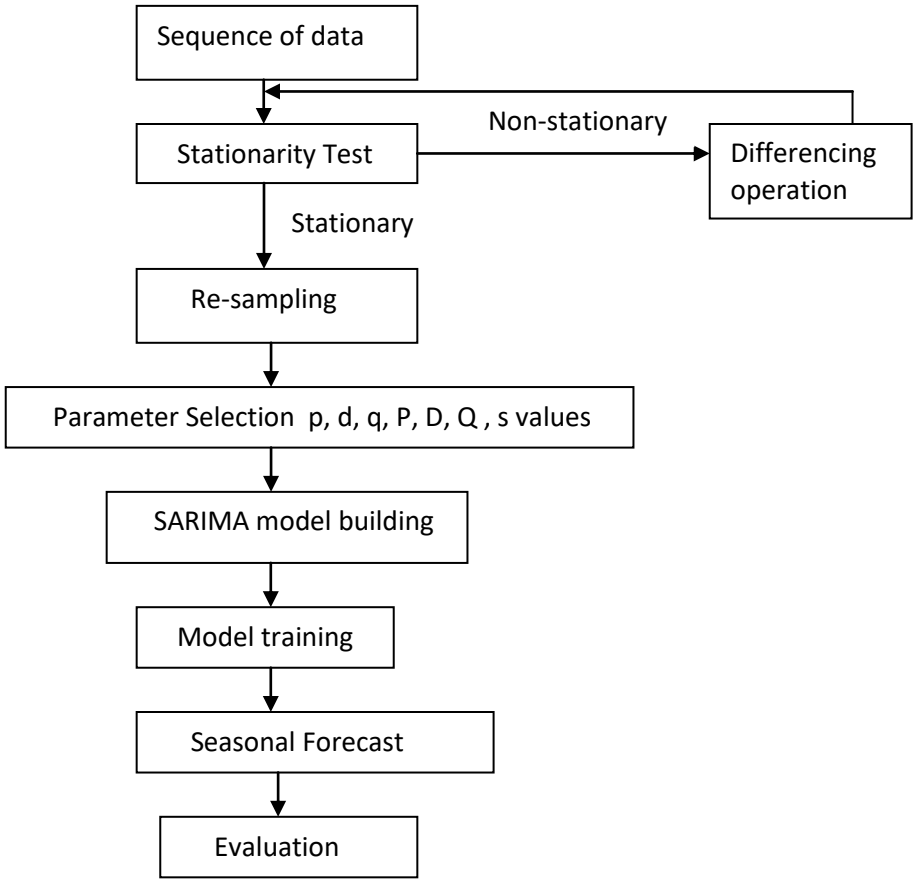
Model Architectures

ARIMA



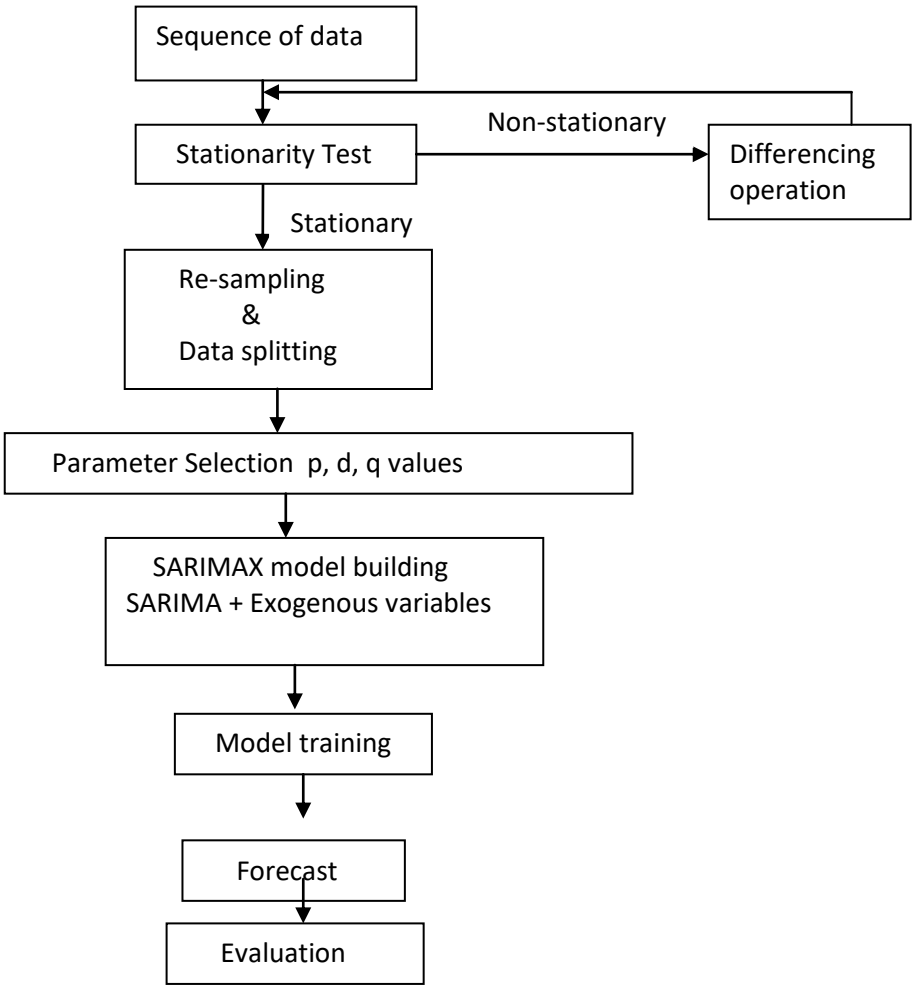
4 System Design and Architecture

SARIMA



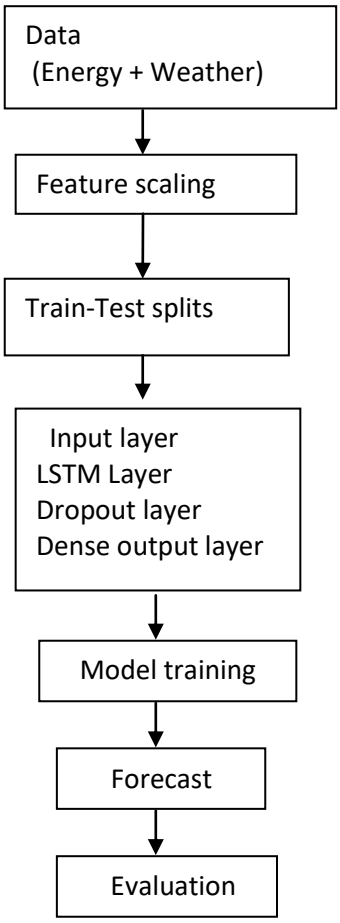
4 System Design and Architecture

SARIMAX



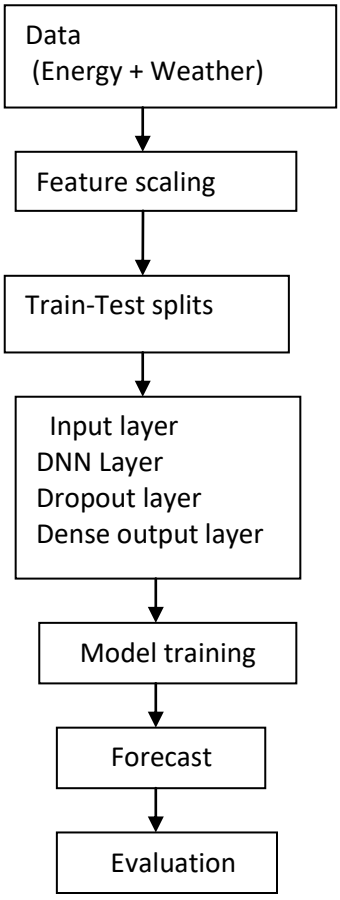
4 System Design and Architecture

LSTM



4 System Design and Architecture

DNN



5 System Development, Testing and Validation

The development phase involves the design and the implementation of both the statistical and neural network models for predicting the electricity production and supply prediction using both the energy production data and the weather data, which acts as the external dependencies.

The complete workflow has divided into following steps:

1. Data Integration: We collected two datasets – one contains the historical data about the energy production and supply over years from 2011-11-24 to 2014-02-27 for energy dataset and 2011-11-11 to 2014-04-10 for weather dataset.

2. Data Preprocessing: We performed different preprocessing ways to clean the data such as handling/imputing missing values, treating outliers, feature engineering, etc. We also performed visualizations to understand the distributions, patterns, trends, acf & pacf plots, correlations, external factors contributions among the data. We performed Principal Component Analysis for dimensionality reduction. And after performing all the above tasks we saved the final dataset in our local drive to use it for further model building.

3. Model Building: We developed five models that are ARIMA, SARIMA, SARIMAX, LSTM, DNN. We developed each model by following their respective architectures, which we described in the section 4.3.

5 System Development, Testing and Validation

Testing and Validation:

The testing phase involves validation the model performance on the unseen data. We split the data chronologically into train (80%) and test (20%) to avoid data leakage as our models are time dependent data.

Coming to validation, we evaluated each model's performance based on key time-series regression metrics:

- Mean Absolute Error (MAE)
- Mean Squared Error (MSE)
- Root Mean Squared Error (RMSE)

These metrics helped us to perform comparative analysis on the performance of the models.

6 Discussion of Results

Model Performance Overview:

- ARIMA:

  Our ARIMA model met most of the conditions needed for a reliable time series model. The residuals are mostly normal, independent, and homoscedastic. Low errors (MAE, MSE, RMSE), which indicates good fit. Stationarity is ensured, and parameters were chosen correctly.

- SARIMA:

  We included the seasonality. It effected the models performance and we noticed fluctuations. High errors when compared with ARIMA model. But, individually performed well with good fit values.

- SARIMAX:

  We introduced the exogenous variables in addition to SARIMA and declared as SARIMAX model. Before that we performed train-test split chronologically. As a result of all the above actions, our SARIMAX model performed well than ARIMA and SARIMA.

- LSTM:

  LSTM was excelled in capturing non-linear trends in temporal data. Our LSTM model managed to generalize well with the unseen data and it provided a good predictive results.

- DNN:

  Our DNN model performed very well on the unseen data. It gave the best results. It predicted well as SARIMAX.

	ARIMA	SARIMA	SARIMAX	LSTM	DNN
MSE	0.0292	0.6968	0.22	0.29	0.17
MAE	0.1265	0.2162	0.08	0.12	0.04
RMSE	0.1710	0.8347	0.28	0.35	0.21

7 Conclusions, Recommendations and Future

Conclusion

The project has aimed to build accurate and efficient models for electricity production and supply prediction by using time series techniques and neural network models by incorporating external features like weather data.

All the classical models ARIMA, SARIMA, SARIMAX and neural network models LSTM, DNN models performed well with the unseen data. But, amongst all time-series models SARIMAX performed well and amongst neural network models DNN performed well. When observed the results individually each and every model got the very less loss values.

Incorporating the weather data to the energy production and supply data, played a crucial role in the entire project. It made the model more practical.

Recommendations

Based on the conclusions and results,

- Data preprocessing steps like stationarity check, treating outliers, handling missing values, concatenating relevant data, dimensionality reduction are very essential for better model performance.
- SARIMAX offers a balanced approach amongst all statistical methods.
- Prediction can be performed live if the data is provided consistently.

7 Conclusions, Recommendations and Future

Future Work

- Inclusion of more external features such as population, etc.
- May work by combining ARIMA-LSTM models as a single model, like combining statistical models with deep learning models.
- Can be experimented with transfer-learning models or attention based architecture models.

References

1. Bryan Lim and Stefan Zohren, "Time-series forecasting with deep learning: a survey", Philosophical Transactions of the Royal Society A, vol. 379, no. 2194, pp. 20200209, 2021.
2. S. Sharma and S. K. Mishra, "Electricity demand estimation using ARIMA forecasting model", 10.1109/MICAI46078.2018.00008, Jan. 2023.
3. Wenxiang Yin; Wen Ye; Guang Liu, “Applying Probabilistic Forecasting of Multidimensional Time Series to Electricity Load Analysis”, 10.1109/SEST61601.2024.10693985, July 2023
4. Yizhuo Liu; Weijia Sun, “ARIMA Time Series Modeling and Forecasting of Enterprise Electric Energy Consumption”, 10.1109/ICFEICT59519.2023.00087, May 2023
5. Y. Wu, P. Cong and Y. Wang, "Charging load forecasting of electric vehicles based on VMD-SSA-SVR", IEEE Transactions on Transportation Electrification, 2023
6. S. Deng, J. Wang, L. Tao, S. Zhang and H. Sun, "EV charging load forecasting model mining algorithm based on hybrid intelligence", Computers and Electrical Engineering, vol. 112, pp. 109010, 2023.

Appendix

1. Energy Data

Figure 1.1

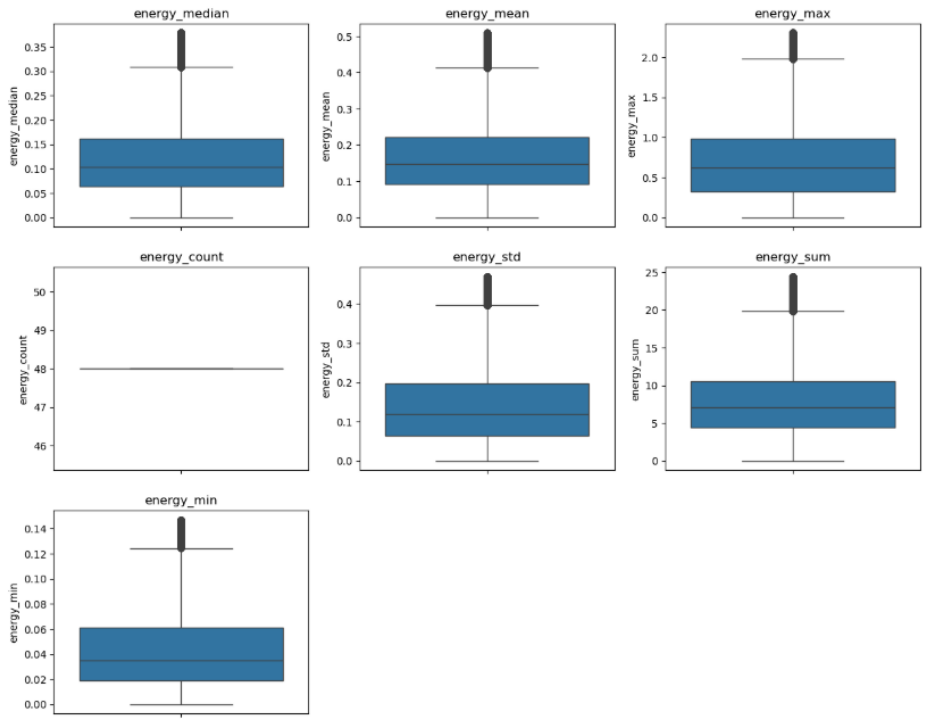
```
import pandas as pd
import numpy as np
from glob import glob
import os

import seaborn as sns
import matplotlib.pyplot as plt

from scipy import stats
```

```
# Load the dataset
daily_data = sorted(glob(r'E:\Project\daily_dataset\daily_dataset/block_*.csv'))
daily_data

# Read and merge all CSV files
daily_data_merge = pd.concat(pd.read_csv(datafile).assign(sourcefilename = datafile)
                             for datafile in daily_data )
daywise_data = daily_data_merge.drop(['sourcefilename'], axis = 1)
```

```
# Save the merged DataFrame to a new CSV file
daywise_data.to_csv("E:/Project/Submissions/Datasets/Energy_merged.csv", index=False)

print(f"Merged CSV saved as: Energy_merged.csv")
```

```
Merged CSV saved as: Energy_merged.csv
```

Figure 1.2



Appendix

Figure 1.3

```
energy_cleaned
```

day	energy_median	energy_mean	energy_max	energy_std	energy_sum	energy_min
2011-11-24	0.2250	0.252812	0.625	0.135267	12.135	0.092
2011-11-24	0.0480	0.048000	0.049	0.000583	2.304	0.047
2011-11-24	0.1250	0.174875	0.611	0.130562	8.394	0.036
2011-11-24	0.0760	0.096250	0.279	0.068798	4.620	0.023
2011-11-24	0.2910	0.297042	0.769	0.112216	14.258	0.127
...	...	...	...	...	...	...
2014-02-27	0.0020	0.009729	0.038	0.012880	0.467	0.000
2014-02-27	0.0820	0.159812	1.161	0.235171	7.671	0.025

```
energy_cleaned = energy_cleaned.reset_index()  # Move 'day' from index to a column
energy_cleaned['day'] = pd.to_datetime(energy_cleaned['day'], errors='coerce')
```

```
# Convert 'day' column to datetime, coercing invalid formats to NaT
energy_cleaned['day'] = pd.to_datetime(energy_cleaned['day'], errors='coerce')
energy_cleaned
```

	day	energy_median	energy_mean	energy_max	energy_std	energy_sum	energy_min
0	2011-11-24	0.2250	0.252812	0.625	0.135267	12.135	0.092
1	2011-11-24	0.0480	0.048000	0.049	0.000583	2.304	0.047
2	2011-11-24	0.1250	0.174875	0.611	0.130562	8.394	0.036
3	2011-11-24	0.0760	0.096250	0.279	0.068798	4.620	0.023
4	2011-11-24	0.2910	0.297042	0.769	0.112216	14.258	0.127
...	...	...	...	...	...	...	...
2949619	2014-02-27	0.0020	0.009729	0.038	0.012880	0.467	0.000
2949620	2014-02-27	0.0820	0.159812	1.161	0.235171	7.671	0.025
2949621	2014-02-27	0.1460	0.211208	0.877	0.228802	10.138	0.023
2949622	2014-02-27	0.0295	0.038000	0.475	0.066159	1.824	0.009
2949623	2014-02-27	0.1905	0.198104	0.768	0.104475	9.509	0.082

2949624 rows × 7 columns

```
# Save the cleaned energy dataset
energy_cleaned.to_csv("E:/Project/Submissions/Datasets/Energy_processed.csv")
print(f"\nProcessed dataset saved as: Energy_processed.csv")
```

```
Processed dataset saved as: Energy_processed.csv
```

2. Weather Dataset

Figure 2.1

```
# Load weather dataset
weather_df = pd.read_csv("E:/Project/Dataset/weather_daily_darksky.csv")

# Check for missing values
print(weather_df.isnull().sum())
```

```
temperatureMax              0
temperatureMaxTime          0
windBearing                 0
icon                        0
dewPoint                    0
temperatureMinTime          0
cloudCover                  1
windSpeed                   0
pressure                    0
apparentTemperatureMinTime  0
apparentTemperatureHigh     0
precipType                  0
visibility                  0
humidity                    0
apparentTemperatureHighTime 0
apparentTemperatureLow      0
apparentTemperatureMax      0
uvIndex                     1
time                        0
sunsetTime                  0
temperatureLow              0
temperatureMin              0
temperatureHigh             0
sunriseTime                 0
temperatureHighTime         0
uvIndexTime                 1
summary                     0
temperatureLowTime          0
apparentTemperatureMin      0
apparentTemperatureMaxTime  0
apparentTemperatureLowTime  0
moonPhase                   0
dtype: int64
```

```
weather_df.columns
```

```
Index(['temperatureMax', 'temperatureMaxTime', 'windBearing', 'icon',
       'dewPoint', 'temperatureMinTime', 'cloudCover', 'windSpeed', 'pressure',
       'apparentTemperatureMinTime', 'apparentTemperatureHigh', 'precipType',
       'visibility', 'humidity', 'apparentTemperatureHighTime',
       'apparentTemperatureLow', 'apparentTemperatureMax', 'uvIndex', 'time',
       'sunsetTime', 'temperatureLow', 'temperatureMin', 'temperatureHigh',
       'sunriseTime', 'temperatureHighTime', 'uvIndexTime', 'summary',
       'temperatureLowTime', 'apparentTemperatureMin',
       'apparentTemperatureMaxTime', 'apparentTemperatureLowTime',
       'moonPhase'],
      dtype='object')
```

Appendix

Figure 2.2

```
weather_final.shape
```

```
(882, 28)
```

```
weather_final.to_csv('E:/Project/Submissions/Datasets/Weather_Processed.csv', index=False)
print(f"\nProcessed dataset saved as: Weather_Processed.csv")
```

```
Processed dataset saved as: Weather_Processed.csv
```

```
weather_final
```

	day	time	temperatureMax	windBearing	dewPoint	cloudCover	windSpeed	pressure	apparentTemperatureHigh	visibility	...	apparentTemperatureMin
0	2011-11-11	23:00:00	11.96	123	9.40	0.79	3.88	1016.08	10.87	3.30	...	6.48
1	2011-11-12	23:00:00	8.59	198	4.49	0.56	3.94	1007.71	5.62	12.09	...	0.11
2	2011-11-13	23:00:00	10.33	225	5.47	0.85	3.54	1032.76	10.33	13.39	...	5.59
3	2011-11-14	23:00:00	8.07	232	3.69	0.32	3.00	1012.12	5.33	11.89	...	0.46
4	2011-11-15	23:00:00	8.22	252	2.79	0.37	4.46	1028.17	5.02	13.16	...	-0.51
...	...	...	...	...	...	...	...	...	...	...	...	...
877	2014-04-06	23:00:00	9.03	233	2.39	0.40	4.55	1002.10	5.99	11.97	...	-1.30
878	2014-04-07	23:00:00	10.31	224	3.08	0.32	4.14	1007.02	10.31	12.04	...	1.41
879	2014-04-08	23:00:00	18.97	172	4.30	0.04	2.78	1022.44	18.97	10.62	...	7.08
880	2014-04-09	23:00:00	8.83	210	1.94	0.59	7.24	994.27	4.73	11.80	...	-1.20
881	2014-04-10	23:00:00	9.90	233	2.95	0.35	9.96	988.63	5.79	12.38	...	1.77

882 rows × 28 columns

3. Final Data

Figure 3.1

```
print(merged_data.dtypes)
```

```
day                         datetime64[ns]
energy_median                      float64
energy_mean                        float64
energy_max                         float64
energy_std                         float64
energy_sum                         float64
energy_min                         float64
time                                object
temperatureMax                     float64
windBearing                          int64
dewPoint                           float64
cloudCover                         float64
windSpeed                          float64
pressure                           float64
apparentTemperatureHigh            float64
visibility                         float64
humidity                           float64
apparentTemperatureLow             float64
apparentTemperatureMax             float64
uvIndex                            float64
time.1                              object
temperatureLow                     float64
temperatureMin                     float64
temperatureHigh                    float64
apparentTemperatureMin             float64
moonPhase                          float64
clear-day                            int32
cloudy                               int32
fog                                  int32
partly-cloudy-day                    int32
partly-cloudy-night                  int32
wind                                 int32
rain                                 int32
snow                                 int32
year                                 int32
month                                int32
date                                 int32
dtype: object
```

```
merged_data.to_csv("E:/Project/Submissions/Datasets/Final_Data.csv", index=False)  # Saves without the index column
print(f"\nProcessed dataset saved as: Final_Data.csv")
```

```
Processed dataset saved as: Final_Data.csv
```

```
merged_data.shape
```

```
(2949624, 37)
```

Appendix

4. Principal Component Analysis

Figure 4.1

```
df = pd.read_csv("E:/Project/Submissions/Datasets/Final_Data.csv")
df
```

	day	energy_median	energy_mean	energy_max	energy_std	energy_sum	energy_min	time	temperatureMax	windBearing	...	cloudy	fog	partly-cloudy-day
0	2011-11-24	0.2250	0.252812	0.625	0.135267	12.135	0.092	23:00:00	15.57	208	...	0	0	1
1	2011-11-24	0.0480	0.048000	0.049	0.000583	2.304	0.047	23:00:00	15.57	208	...	0	0	1
2	2011-11-24	0.1250	0.174875	0.611	0.130562	8.394	0.036	23:00:00	15.57	208	...	0	0	1
3	2011-11-24	0.0760	0.096250	0.279	0.068798	4.620	0.023	23:00:00	15.57	208	...	0	0	1
4	2011-11-24	0.2910	0.297042	0.769	0.112216	14.258	0.127	23:00:00	15.57	208	...	0	0	1
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
2949619	2014-02-27	0.0020	0.009729	0.038	0.012880	0.467	0.000	23:00:00	9.21	302	...	0	0	1
2949620	2014-02-27	0.0820	0.159812	1.161	0.235171	7.671	0.025	23:00:00	9.21	302	...	0	0	1
2949621	2014-02-27	0.1460	0.211208	0.877	0.228802	10.138	0.023	23:00:00	9.21	302	...	0	0	1
2949622	2014-02-27	0.0295	0.038000	0.475	0.066159	1.824	0.009	23:00:00	9.21	302	...	0	0	1
2949623	2014-02-27	0.1905	0.198104	0.768	0.104475	9.509	0.082	23:00:00	9.21	302	...	0	0	1

2949624 rows × 37 columns

```
df.columns
```

```
Index(['day', 'energy_median', 'energy_mean', 'energy_max', 'energy_std',
       'energy_sum', 'energy_min', 'time', 'temperatureMax', 'windBearing',
       'dewPoint', 'cloudCover', 'windSpeed', 'pressure',
       'apparentTemperatureHigh', 'visibility', 'humidity',
       'apparentTemperatureLow', 'apparentTemperatureMax', 'uvIndex', 'time.1',
       'temperatureLow', 'temperatureMin', 'temperatureHigh',
       'apparentTemperatureMin', 'moonPhase', 'clear-day', 'cloudy', 'fog',
       'partly-cloudy-day', 'partly-cloudy-night', 'wind', 'rain', 'snow',
       'year', 'month', 'date'],
      dtype='object')
```

Figure 4.2

```
from sklearn.decomposition import PCA
pca = PCA()
pca.fit(X_scaled)

plt.figure(1, figsize=(8, 7))
plt.clf()  # keeping the visualization clean
plt.plot(pca.explained_variance_ratio_, linewidth=2)

# explained_variance_ratio_ is the proportion of variance explained by each of the principal components (PCs) extracted from the data
plt.xlabel('n_components')
plt.ylabel('explained_variance_ratio_')

# performing PCA

#pca = PCA(n_components = 5)
#X_pca = pca.fit_transform(X_scaled)

# Apply PCA
pca = PCA(n_components=0.95)  # Retain 95% variance
X_pca = pca.fit_transform(X_scaled)

print(f"Number of Principal Components Selected: {pca.n_components_}")

# Explained variance ratio
explained_variance = np.cumsum(pca.explained_variance_ratio_)
print("Explained Variance Ratio:", explained_variance)

# Convert PCA transformed data back to DataFrame
df_pca = pd.DataFrame(X_pca, columns=[f"PC{i+1}" for i in range(pca.n_components_)])
```

```
Number of Principal Components Selected: 17
Explained Variance Ratio: [0.27727587 0.40852748 0.48421756 0.55213036 0.60437583 0.65174788
 0.69487501 0.7285978  0.76127885 0.79256162 0.8221334  0.85138751
 0.87791799 0.90172317 0.92229907 0.93758725 0.95185476]
```

Appendix

Figure 4.3

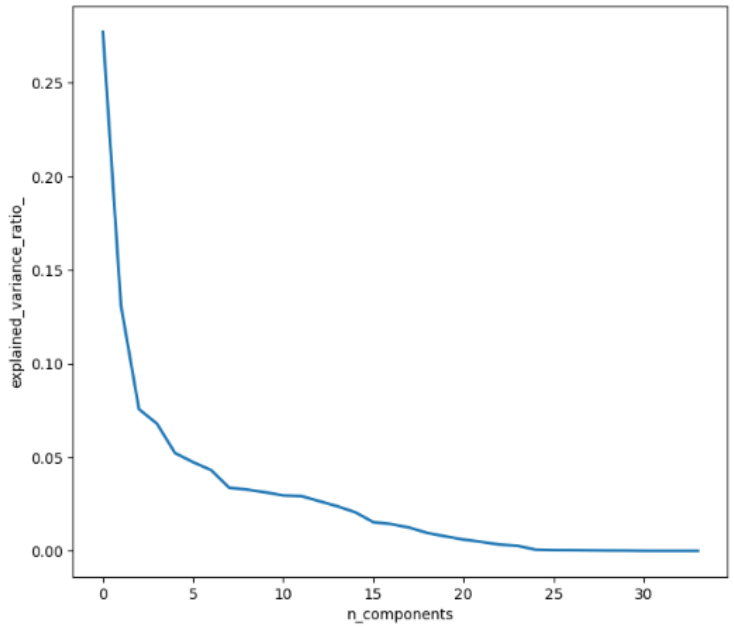
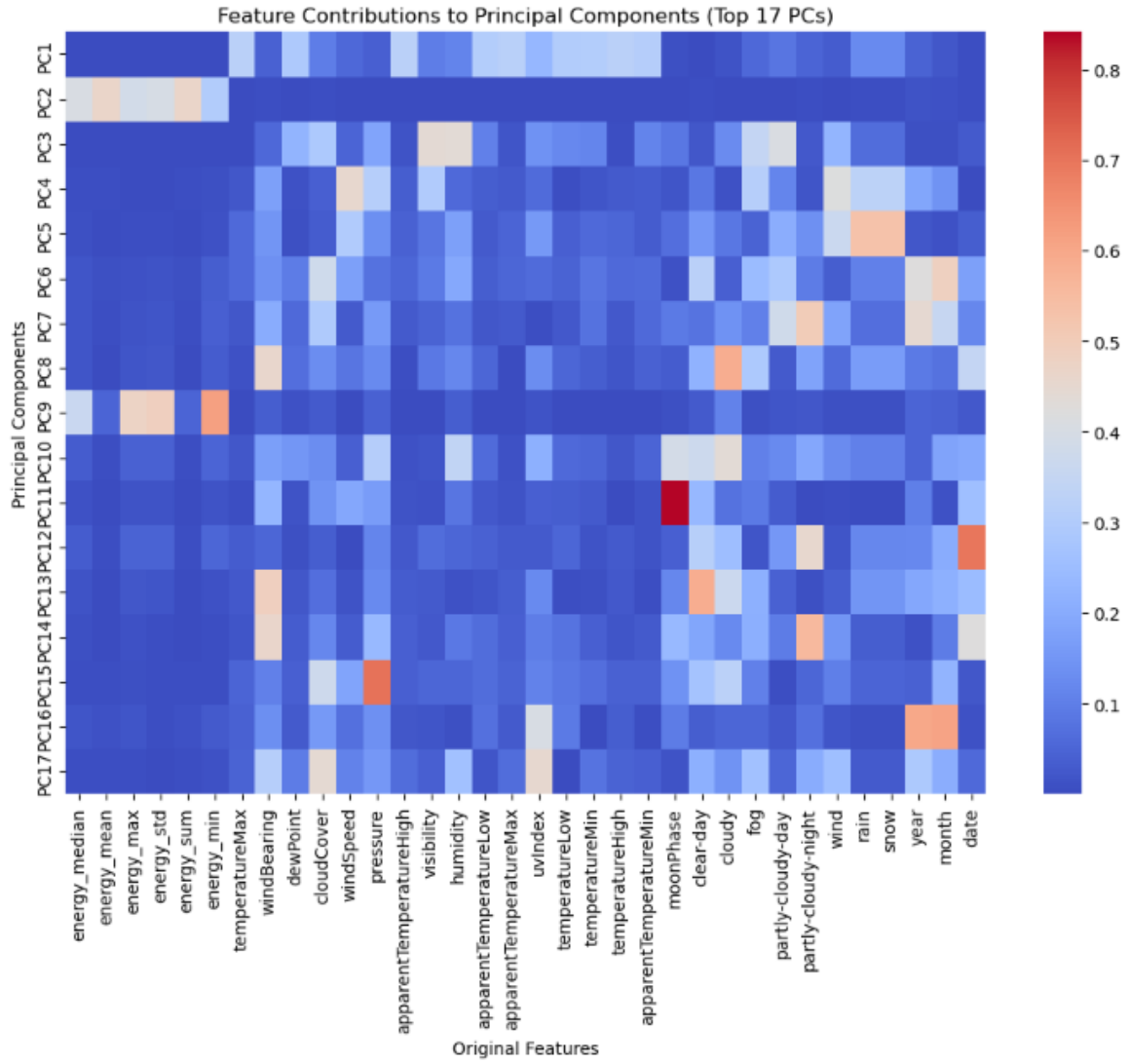


Figure 4.4

```python
# Assuming `pca` is your trained PCA model and `df` is your original dataset
loading_matrix = pd.DataFrame(
    abs(pca.components_),  # Take absolute values to get the strength of influence
    columns=df.columns  # Retain original feature names
)

# Select the first 17 principal components
loading_matrix_17 = loading_matrix.iloc[:17, :]

# Plot heatmap
plt.figure(figsize=(12, 8))
sns.heatmap(loading_matrix_17, cmap="coolwarm", annot=False, xticklabels=True, yticklabels=[f"PC{i+1}" for i in range(17)])
plt.xlabel("Original Features")
plt.ylabel("Principal Components")
plt.title("Feature Contributions to Principal Components (Top 17 PCs)")
plt.xticks(rotation=90)
plt.show()
```

Appendix

Figure 4.5



Appendix

5. ARIMA

Figure 5.1

```
df.columns
```

```
Index(['day', 'energy_median', 'energy_mean', 'energy_max', 'energy_std',
       'energy_sum', 'energy_min', 'time', 'temperatureMax', 'windBearing',
       'dewPoint', 'cloudCover', 'windSpeed', 'pressure',
       'apparentTemperatureHigh', 'visibility', 'humidity',
       'apparentTemperatureLow', 'apparentTemperatureMax', 'uvIndex', 'time.1',
       'temperatureLow', 'temperatureMin', 'temperatureHigh',
       'apparentTemperatureMin', 'moonPhase', 'clear-day', 'cloudy', 'fog',
       'partly-cloudy-day', 'partly-cloudy-night', 'wind', 'rain', 'snow',
       'year', 'month', 'date'],
      dtype='object')
```

```
# Load dataset (ensure 'day' is the index)
df['day'] = pd.to_datetime(df['day'])
df.set_index('day', inplace=True)
```

```
# Selecting target variable (energy_sum as an example)
target_col = 'energy_sum'
df_re = df[target_col]
```

```
df_resampled = df_re.resample('W').mean()  # Resampling to daily mean if your index is datetime
plt.figure(figsize=(12, 5))
plt.plot(df_resampled, label="Resampled Time Series")
plt.title("Time Series Plot (Resampled)")
plt.legend()
plt.show()
```

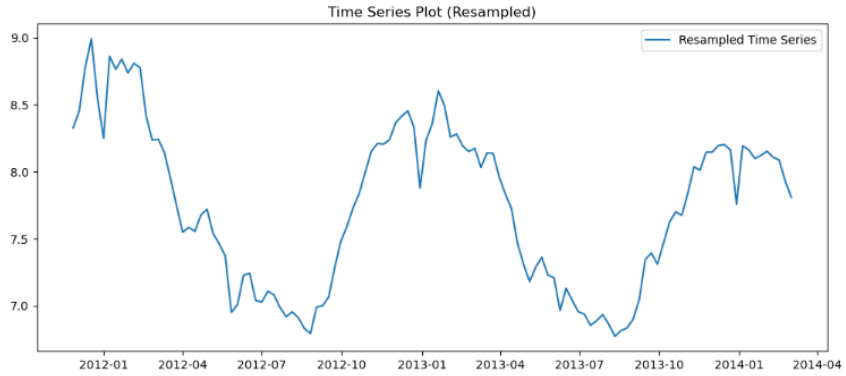
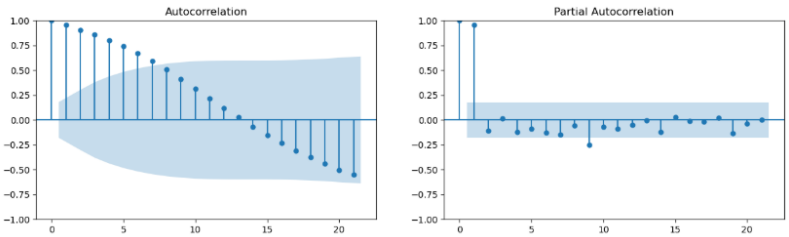


Figure 5.2

```
# Function to perform ADF test
def adf_test(series):
    result = adfuller(series)
    print(f"ADF Statistic: {result[0]}")
    print(f"p-value: {result[1]}")
    if result[1] <= 0.05:
        print("Data is stationary.")
    else:
        print("Data is non-stationary.")

# Perform ADF test on the sample
adf_test(df_resampled)
```

```
ADF Statistic: -3.871216634746527
p-value: 0.002257190357061092
Data is stationary.
```

```
# Plot ACF & PACF (Identify AR & MA terms)

from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

fig, axes = plt.subplots(1, 2, figsize=(15, 4))
plot_acf(df_resampled, ax=axes[0])  # Identify MA component
plot_pacf(df_resampled, ax=axes[1])  # Identify AR component
plt.show()
```



Appendix

Figure 5.3

```
# Find Best ARIMA Parameters (p, d, q)

auto_arima_model = auto_arima(df_resampled, seasonal=False, stepwise=True, trace=True)
print(auto_arima_model.summary())
```

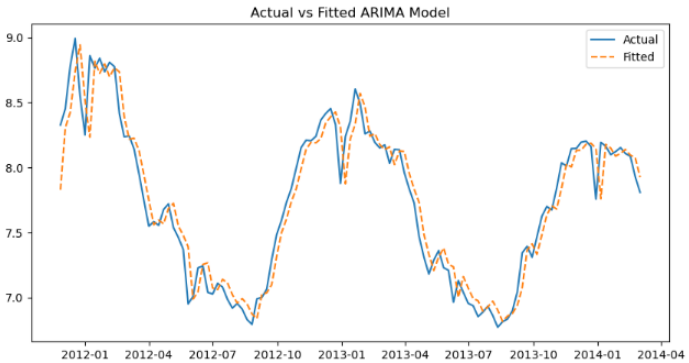
```
Performing stepwise search to minimize aic
 ARIMA(2,0,2)(0,0,0)[0]             : AIC=-71.944, Time=0.26 sec
 ARIMA(0,0,0)(0,0,0)[0]             : AIC=827.411, Time=0.02 sec
 ARIMA(1,0,0)(0,0,0)[0]             : AIC=inf, Time=0.05 sec
 ARIMA(0,0,1)(0,0,0)[0]             : AIC=inf, Time=0.06 sec
 ARIMA(1,0,2)(0,0,0)[0]             : AIC=-73.824, Time=0.12 sec
 ARIMA(0,0,2)(0,0,0)[0]             : AIC=inf, Time=0.19 sec
 ARIMA(1,0,1)(0,0,0)[0]             : AIC=-74.818, Time=0.09 sec
 ARIMA(2,0,1)(0,0,0)[0]             : AIC=-73.632, Time=0.19 sec
 ARIMA(2,0,0)(0,0,0)[0]             : AIC=inf, Time=0.12 sec
 ARIMA(1,0,1)(0,0,0)[0] intercept   : AIC=-81.437, Time=0.10 sec
 ARIMA(0,0,1)(0,0,0)[0] intercept   : AIC=94.214, Time=0.05 sec
 ARIMA(1,0,0)(0,0,0)[0] intercept   : AIC=-82.241, Time=0.05 sec
 ARIMA(0,0,0)(0,0,0)[0] intercept   : AIC=218.271, Time=0.04 sec
 ARIMA(2,0,0)(0,0,0)[0] intercept   : AIC=-81.241, Time=0.10 sec
 ARIMA(2,0,1)(0,0,0)[0] intercept   : AIC=-80.080, Time=0.26 sec

Best model:  ARIMA(1,0,0)(0,0,0)[0] intercept
Total fit time: 1.715 seconds
```

Figure 5.4

```
# Train the ARIMA Model
# Using the best (p, d, q) values from auto_arima

p, d, q = auto_arima_model.order  # Get optimal parameters

# Fit ARIMA model
model = ARIMA(df_resampled, order=(p, d, q))
#model = ARIMA(df_resampled, order=(1,1,1))
#model = ARIMA(df_resampled, order=(1,1,0))
model_fit = model.fit()

# Summary of the model
print(model_fit.summary())
```

Figure 5.5

```
# In-sample Predictions (Fitting on Training Data)

series_pred = model_fit.fittedvalues

plt.figure(figsize=(10, 5))
plt.plot(df_resampled, label="Actual")
plt.plot(series_pred, label="Fitted", linestyle="dashed")
plt.title("Actual vs Fitted ARIMA Model")
plt.legend()
plt.show()
```



Appendix

Figure 5.6

```
# Forecast Future Values

future_steps = 12  # Forecast next 30 days
forecast = model_fit.forecast(steps=future_steps)

plt.figure(figsize=(10, 5))
plt.plot(df_resampled, label="Past Data")
plt.plot(pd.date_range(df_resampled.index[-1], periods=future_steps, freq='W'), forecast, label="Forecast", linestyle="dashed")
plt.title("Future Forecast (ARIMA)")
plt.legend()
plt.show()
```

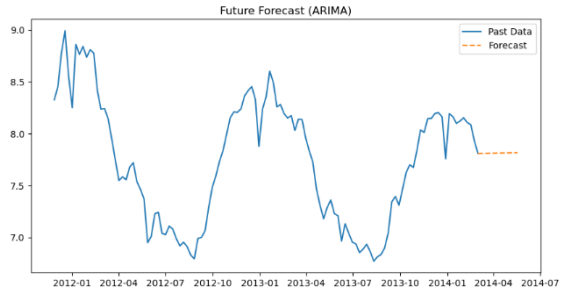
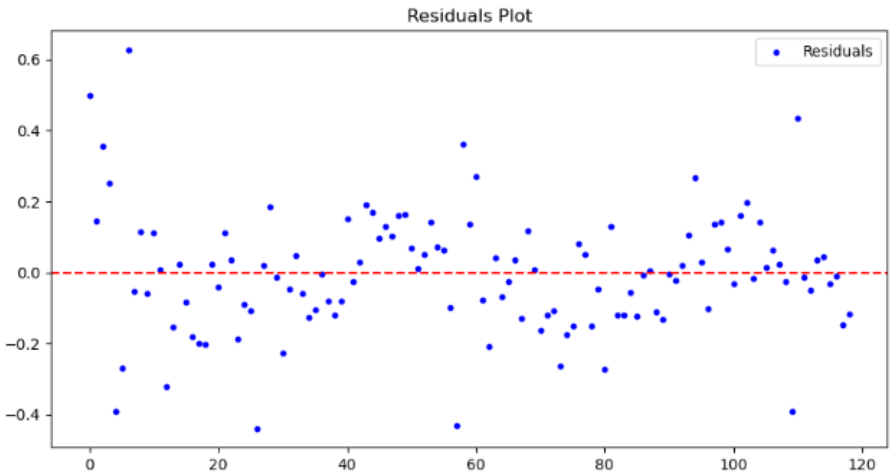


Figure 5.7

```
# Evaluate the Model

from sklearn.metrics import mean_absolute_error, mean_squared_error

y_true = df_resampled[d:]  # Adjust for differencing (d)
y_pred = series_pred[d:]

mae = mean_absolute_error(y_true, y_pred)
mse = mean_squared_error(y_true, y_pred)
rmse = np.sqrt(mse)

print(f"MAE: {mae:.4f}")
print(f"MSE: {mse:.4f}")
print(f"RMSE: {rmse:.4f}")
```

```
MAE: 0.1265
MSE: 0.0292
RMSE: 0.1710
```

Figure 5.8



Appendix

6. SARIMA

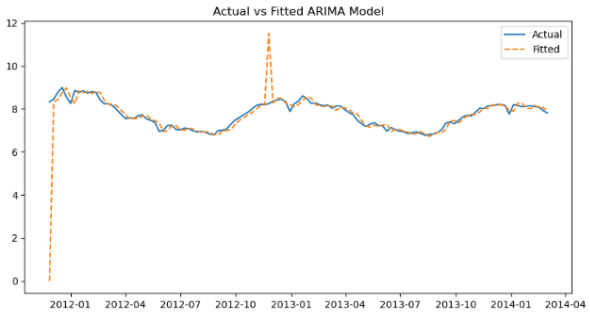
Figure 6.1

```
stepwise_fit = auto_arima(df_resampled, seasonal=True, m=52, trace=True, suppress_warnings=True)
print(stepwise_fit.summary())

p,d,q = stepwise_fit.order

# Define the SARIMA model (modify p, d, q, P, D, Q based on ACF/PACF)
model = SARIMAX(df_resampled,
                order=(1, 0, 1),  # (p,d,q) - Modify based on ACF/PACF
                seasonal_order=(1, 0, 1, 52),  # (P,D,Q,m) - m=52 for weekly seasonality
                enforce_stationarity=False,
                enforce_invertibility=False)

# Fit the model
model_fit = model.fit(disp=False)

# Print model summary
print(model_fit.summary())
```

```
Performing stepwise search to minimize aic
 ARIMA(2,0,2)(1,0,1)[52] intercept   : AIC=-101.699, Time=7.66 sec
 ARIMA(0,0,0)(0,0,0)[52] intercept   : AIC=218.271, Time=0.06 sec
 ARIMA(1,0,0)(1,0,0)[52] intercept   : AIC=-100.151, Time=5.46 sec
 ARIMA(0,0,1)(0,0,1)[52] intercept   : AIC=inf, Time=2.77 sec
 ARIMA(0,0,0)(0,0,0)[52]             : AIC=827.411, Time=0.03 sec
 ARIMA(2,0,2)(0,0,1)[52] intercept   : AIC=-93.742, Time=4.88 sec
 ARIMA(2,0,2)(1,0,0)[52] intercept   : AIC=-75.830, Time=6.93 sec
 ARIMA(2,0,2)(2,0,1)[52] intercept   : AIC=-99.764, Time=30.87 sec
 ARIMA(2,0,2)(1,0,2)[52] intercept   : AIC=inf, Time=26.20 sec
 ARIMA(2,0,2)(0,0,0)[52] intercept   : AIC=-78.178, Time=0.54 sec
 ARIMA(2,0,2)(0,0,2)[52] intercept   : AIC=inf, Time=23.40 sec
 ARIMA(2,0,2)(2,0,0)[52] intercept   : AIC=-101.740, Time=35.43 sec
 ARIMA(1,0,2)(2,0,0)[52] intercept   : AIC=-103.767, Time=43.63 sec
 ARIMA(1,0,2)(1,0,0)[52] intercept   : AIC=-42.574, Time=26.36 sec
 ARIMA(1,0,2)(2,0,1)[52] intercept   : AIC=-101.965, Time=41.19 sec
 ARIMA(1,0,2)(1,0,1)[52] intercept   : AIC=-103.883, Time=14.87 sec
 ARIMA(1,0,2)(0,0,1)[52] intercept   : AIC=-95.700, Time=4.15 sec
 ARIMA(1,0,2)(1,0,2)[52] intercept   : AIC=inf, Time=51.81 sec
 ARIMA(1,0,2)(0,0,0)[52] intercept   : AIC=-80.035, Time=0.50 sec
 ARIMA(1,0,2)(0,0,2)[52] intercept   : AIC=inf, Time=23.29 sec
 ARIMA(1,0,2)(2,0,2)[52] intercept   : AIC=-99.974, Time=38.95 sec
 ARIMA(0,0,2)(1,0,1)[52] intercept   : AIC=inf, Time=6.36 sec
 ARIMA(1,0,1)(1,0,1)[52] intercept   : AIC=inf, Time=7.10 sec
 ARIMA(1,0,3)(1,0,1)[52] intercept   : AIC=-101.995, Time=8.00 sec
 ARIMA(0,0,1)(1,0,1)[52] intercept   : AIC=inf, Time=5.80 sec
 ARIMA(0,0,3)(1,0,1)[52] intercept   : AIC=inf, Time=9.00 sec
 ARIMA(2,0,1)(1,0,1)[52] intercept   : AIC=-102.627, Time=9.93 sec
 ARIMA(2,0,3)(1,0,1)[52] intercept   : AIC=-101.221, Time=8.58 sec
 ARIMA(1,0,2)(1,0,1)[52]             : AIC=inf, Time=6.60 sec
```

Figure 6.2

```
series_pred = model_fit.fittedvalues

plt.figure(figsize=(10, 5))
plt.plot(df_resampled, label="Actual")
plt.plot(series_pred, label="Fitted", linestyle="dashed")
plt.title("Actual vs Fitted ARIMA Model")
plt.legend()
plt.show()
```



Appendix

Figure 6.3

```
# Forecast next 12 weeks
forecast_steps = 12
forecast = model_fit.get_forecast(steps=forecast_steps)
forecast_index = pd.date_range(start=df_resampled.index[-1], periods=forecast_steps+1, freq='W')[1:]

# Get confidence intervals
forecast_conf_int = forecast.conf_int()

# Plot results
plt.figure(figsize=(12, 6))
plt.plot(df_resampled, label="Observed")
plt.plot(forecast_index, forecast.predicted_mean, label="Forecast", color='red')
plt.fill_between(forecast_index, forecast_conf_int.iloc[:, 0], forecast_conf_int.iloc[:, 1], color='pink', alpha=0.3)
plt.legend()
plt.title("SARIMA Forecast")
plt.show()
```

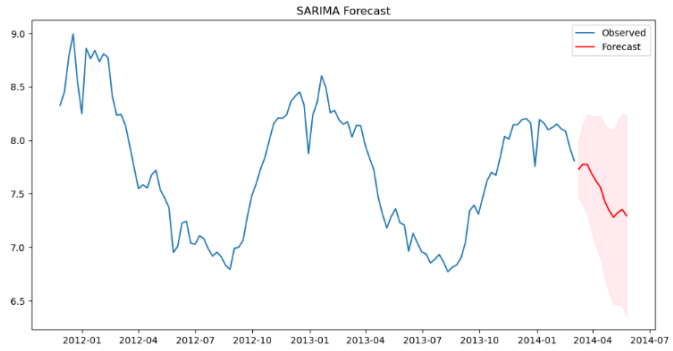


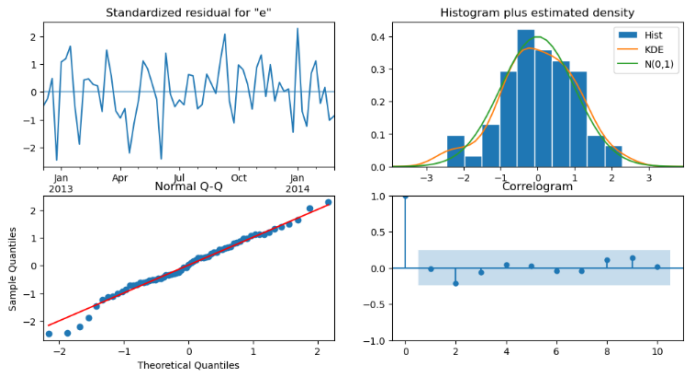
Figure 6.4

```
# Evaluate the Model

y_true = df_resampled[0:]  # Adjust for differencing (d)
y_pred = series_pred[0:]

mae = mean_absolute_error(y_true, y_pred)
mse = mean_squared_error(y_true, y_pred)
rmse = np.sqrt(mse)

print(f"MAE: {mae:.4f}")
print(f"MSE: {mse:.4f}")
print(f"RMSE: {rmse:.4f}")
```

```
MAE: 0.2162
MSE: 0.6967
RMSE: 0.8347
```

Figure 6.5

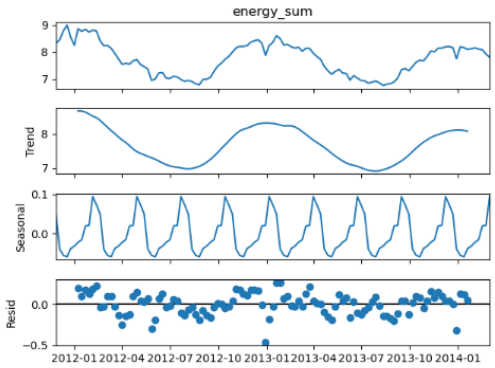
```
# Diagnostic plots
model_fit.plot_diagnostics(figsize=(12, 6))
plt.show()
```



Appendix

Figure 6.6

```
from statsmodels.tsa.seasonal import seasonal_decompose
import matplotlib.pyplot as plt

y = df["energy_sum"]

result = seasonal_decompose(df_resampled, model="additive", period=12)  # Change 'period' if needed
result.plot()
plt.show()
```



7. SARIMAX

Figure 7.1

```
# Select exogenous variables (excluding target 'energy_sum' and 'date' column)
exog_cols = ['month', 'partly-cloudy-night', 'cloudy', 'temperatureHigh',
             'visibility']# 'windSpeed', 'rain', 'moonPhase', 'cLear-day', 'pressure', 'uvIndex']

df_exog = df[exog_cols].resample('W').mean().dropna()
```

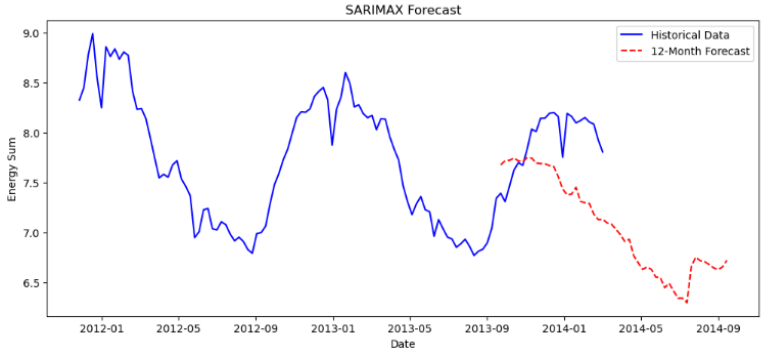
```
# Ensure exogenous variables and target have the same index
df_exog = df_exog.loc[df_resampled.index]
```

```
#time-based (chronological) split, meaning:
#The first 80% of the data is used for training.
#The last 20% is used for testing.

# Train-Test Split (80% train, 20% test)
train_size = int(len(df_resampled) * 0.8)
train_y, test_y = df_resampled[:train_size], df_resampled[train_size:]
train_exog, test_exog = df_exog[:train_size], df_exog[train_size:]
```

Figure 7.2

```
steps_ahead = 52  # 12 months for weekly data
future_forecast = sarimax_result.forecast(steps=steps_ahead, exog=future_exog)
```

```
plt.figure(figsize=(12, 5))
plt.plot(df_resampled, label="Historical Data", color='blue')
plt.plot(future_forecast, label="12-Month Forecast", color='red', linestyle='dashed')
plt.xlabel("Date")
plt.ylabel("Energy Sum")
plt.title("SARIMAX Forecast")
plt.legend()
plt.show()
```



Appendix

8. LSTM

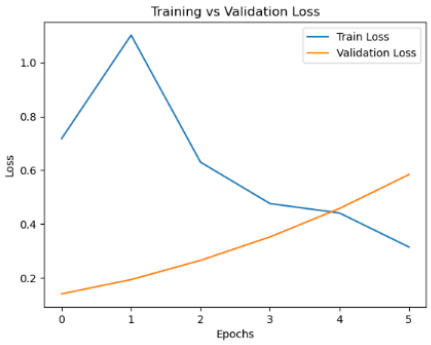
Figure 8.1

```
model = Sequential()

# First LSTM layer with L2 regularization
model.add(LSTM(50, return_sequences=True, kernel_regularizer=l2(0.001), input_shape=(30, 1)))
model.add(Dropout(0.3))  # Increase dropout slightly

# Second LSTM layer with L2 regularization
model.add(LSTM(50, return_sequences=False, kernel_regularizer=l2(0.001)))
model.add(Dropout(0.3))

# Output layer
model.add(Dense(1))

model.compile(optimizer='adam', loss='mean_squared_error')

early_stopping = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)

# Train the model
history = model.fit(X_train, y_train, epochs=50, batch_size=32, validation_split=0.2, callbacks=[early_stopping])
```

```
Epoch 1/50
C:\Users\ABCD\anaconda3\Lib\site-packages\keras\src\layers\rnn\rnn.py:200: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. W
hen using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(**kwargs)
1/1 ━━━━━━━━━━━━━━━━━━━━ 4s 4s/step - loss: 0.7179 - val_loss: 0.1409
Epoch 2/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 132ms/step - loss: 1.1022 - val_loss: 0.1940
Epoch 3/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 123ms/step - loss: 0.6301 - val_loss: 0.2655
Epoch 4/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 149ms/step - loss: 0.4764 - val_loss: 0.3526
Epoch 5/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 115ms/step - loss: 0.4414 - val_loss: 0.4580
Epoch 6/50
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 135ms/step - loss: 0.3151 - val_loss: 0.5843
```

Figure 8.2

```
import matplotlib.pyplot as plt

# Assuming 'history' is the object returned by model.fit()
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.title("Training vs Validation Loss")
plt.show()
```



Appendix

9. DNN

Figure 9.1

```
# Build DNN model with more layers
model = Sequential([
    Dense(256, activation='relu', input_shape=(X_train.shape[1],)),  # Increased neurons
    Dropout(0.3),  # Increased dropout for regularization
    Dense(128, activation='relu'),
    Dropout(0.3),
    Dense(64, activation='relu'),
    Dropout(0.2),
    Dense(32, activation='relu'),
    Dense(16, activation='relu'),  # Additional layer
    Dense(1)  # Output layer (forecasted value)
])

# Compile model
model.compile(optimizer='sgd', loss='mean_squared_error', metrics=['mae'])
#model.compile(optimizer='adam', loss='mse', metrics=['mae'])
# model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

early_stopping = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)

# Train model
history = model.fit(X_train, y_train, epochs=10, batch_size=32, validation_data=(X_test, y_test), callbacks=[early_stopping])
```

```
C:\Users\ABCD\anaconda3\Lib\site-packages\keras\src\layers\core\dense.py:87: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer.
When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/10
73740/73740 ━━━━━━━━━━━━━━━━━━━━ 390s 5ms/step - loss: 0.0382 - mae: 0.1582 - val_loss: 0.0430 - val_mae: 0.1651
Epoch 2/10
73740/73740 ━━━━━━━━━━━━━━━━━━━━ 364s 5ms/step - loss: 0.0378 - mae: 0.1574 - val_loss: 0.0424 - val_mae: 0.1651
Epoch 3/10
73740/73740 ━━━━━━━━━━━━━━━━━━━━ 381s 5ms/step - loss: 0.0377 - mae: 0.1573 - val_loss: 0.0426 - val_mae: 0.1650
Epoch 4/10
73740/73740 ━━━━━━━━━━━━━━━━━━━━ 358s 5ms/step - loss: 0.0377 - mae: 0.1572 - val_loss: 0.0429 - val_mae: 0.1651
Epoch 5/10
73740/73740 ━━━━━━━━━━━━━━━━━━━━ 248s 3ms/step - loss: 0.0376 - mae: 0.1570 - val_loss: 0.0422 - val_mae: 0.1652
Epoch 6/10
73740/73740 ━━━━━━━━━━━━━━━━━━━━ 226s 3ms/step - loss: 0.0377 - mae: 0.1572 - val_loss: 0.0427 - val_mae: 0.1651
Epoch 7/10
73740/73740 ━━━━━━━━━━━━━━━━━━━━ 277s 4ms/step - loss: 0.0376 - mae: 0.1570 - val_loss: 0.0428 - val_mae: 0.1651
Epoch 8/10
73740/73740 ━━━━━━━━━━━━━━━━━━━━ 307s 4ms/step - loss: 0.0376 - mae: 0.1572 - val_loss: 0.0425 - val_mae: 0.1651
Epoch 9/10
73740/73740 ━━━━━━━━━━━━━━━━━━━━ 252s 3ms/step - loss: 0.0376 - mae: 0.1571 - val_loss: 0.0430 - val_mae: 0.1651
Epoch 10/10
73740/73740 ━━━━━━━━━━━━━━━━━━━━ 250s 3ms/step - loss: 0.0376 - mae: 0.1570 - val_loss: 0.0425 - val_mae: 0.1651
```

ORIGINALITY REPORT

15%	12%	8%	7%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	de.overleaf.com Internet Source	3%
2	www.coursehero.com Internet Source	1%
3	Submitted to Loughborough University Student Paper	1%
4	R. N. V. Jagan Mohan, B. H. V. S. Rama Krishnam Raju, V. Chandra Sekhar, T. V. K. P. Prasad. "Algorithms in Advanced Artificial Intelligence - Proceedings of International Conference on Algorithms in Advanced Artificial Intelligence (ICAAAI-2024)", CRC Press, 2025 Publication	1%
5	arxiv.org Internet Source	1%
6	su-plus.strathmore.edu Internet Source	1%
7	David H. Hopfe, Kiljae Lee, Chunyan Yu. "Short-term forecasting airport passenger flow during periods of volatility: Comparative investigation of time series vs. neural network	1%

	models", Journal of Air Transport Management, 2024 Publication	
8	thesai.org Internet Source	1 %
9	Submitted to Sunway Education Group Student Paper	1 %
10	laccei.org Internet Source	<1 %
11	www.mdpi.com Internet Source	<1 %
12	Submitted to universititeknologimara Student Paper	<1 %
13	arno.uvt.nl Internet Source	<1 %
14	doctorpenguin.com Internet Source	<1 %
15	vugene.com Internet Source	<1 %
16	archpublichealth.biomedcentral.com Internet Source	<1 %
17	stud.epsilon.slu.se Internet Source	<1 %
18	Submitted to Liverpool John Moores University Student Paper	<1 %
19	Renxue Shang, Yongjun Ma. "Electric Vehicle Charging Load Forecasting Based on K-	<1 %

	Means++-GRU-KSVR", World Electric Vehicle Journal, 2024 Publication	
20	Ton Duc Thang University Publication	<1 %
21	assets-eu.researchsquare.com Internet Source	<1 %
22	www.fluoridealert.org Internet Source	<1 %
23	docshare.tips Internet Source	<1 %
24	ijarsct.co.in Internet Source	<1 %
25	isset.shiksha.com Internet Source	<1 %
26	David Medall Martos. "Numerical and experimental research on the fire resistance of composite columns with steel sections embedded in concrete and high strength materials", Universitat Politecnica de Valencia, 2024 Publication	<1 %
27	Edelberg, Thomas. "Rural School District Leadership: Supporting Technology Integration for Instructional Spaces.", Indiana University, 2020 Publication	<1 %
28	Guo, Changmiao. "Structural Basis for Interactions of Microtubule-Associated	<1 %

	Proteins with Microtubules by Magic Angle Spinning NMR Spectroscopy and Methodological Developments for Biomolecular Applications.", University of Delaware, 2020 Publication	
29	Khalilalrahman Dehvari, Kai Yi Liu, Po-Jen Tseng, Gangaraju Gedda, Wubshet Mekonnen Girma, Jia-Yaw Chang. "Sonochemical-assisted green synthesis of nitrogen-doped carbon dots from crab shell as targeted nanoprobes for cell imaging", Journal of the Taiwan Institute of Chemical Engineers, 2018 Publication	<1 %
30	dl.lib.uom.lk Internet Source	<1 %
31	ejournal.bumipublikasinusantara.id Internet Source	<1 %
32	mdpi-res.com Internet Source	<1 %
33	encyclopedia.pub Internet Source	<1 %
34	tenerife.chat Internet Source	<1 %
35	www.i-scholar.in Internet Source	<1 %
36	www.learninsta.com Internet Source	<1 %
37	Submitted to PSG Institutions Student Paper	

<1%

Exclude quotes	On	Exclude matches	< 4 words
Exclude bibliography	On		

GITAM
DEEMED TO BE UNIVERSITY